Output of ls -R lab06 | head -40, showing the layout from Part 0.

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# cd ..
(.venv) root@e4d554fa1453:/ITMD544/project# ls -R lab06 | head -40
lab06:
advisor-app
chaos_test_uploader_35206.csv
chaos_test_uploader_36057.csv
chaos_test_viewer_36682.csv
create_app_stack.sh.bak
create_app_stack.sh.orig
create_database.sh
create_weather_stack.sh
delete_app_stack.sh.orig
delete_database.sh
delete_weather_stack.sh
launch-template.json
launch-template.json.bak
previous
pytest.ini
requirements.txt
stack
testing
tests
uploader_instance_id.txt
viewer_instance_id.txt
weather-app

lab06/advisor-app:
app.js
package.json

lab06/previous:
create_app_stack.sh
create_stack.py
delete_app_stack.sh
delete_stack.py
uploader-app
viewer-app

lab06/previous/uploader-app:
app.js
```

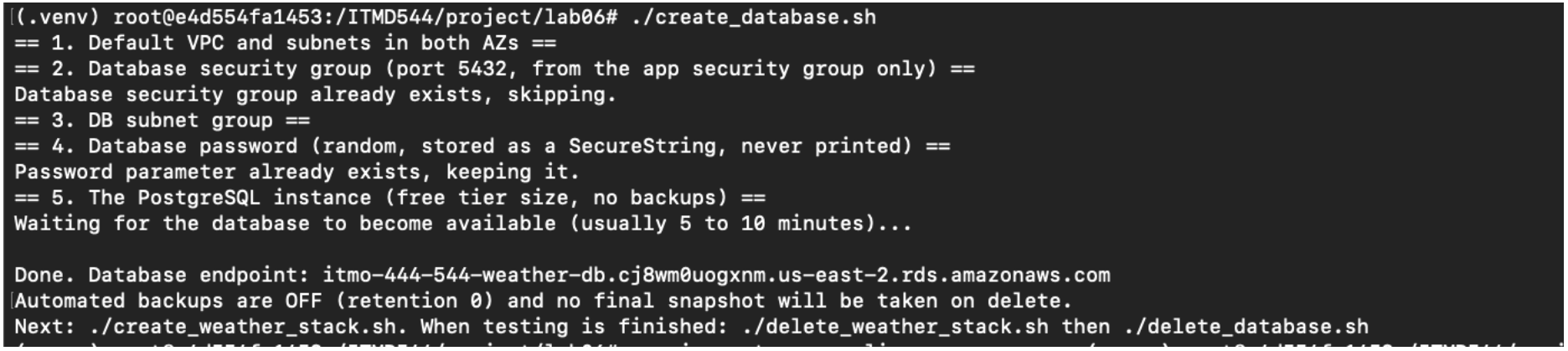
Output of the NWS calls from Part A and one sentence on why two upstream calls are needed.

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s https://api.zippopotam.us/us/60616 | python3 -m json.tool
{
    "country": "United States",
    "country abbreviation": "US",
    "post code": "60616",
    "places": [
        {
            "place name": "Chicago",
            "longitude": "-87.6306",
            "latitude": "41.8426",
            "state": "Illinois",
            "state abbreviation": "IL"
        }
    ]
}
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s -o /dev/null -w "%{http_code}\n" https://api.zippopotam.us/us/00000
404
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# UA="(itmo-444-544 weather lab, you@example.com)"
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# UA="(itmo-444-544 weather lab, jshen25@hawk.illinoistech.edu)"
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s -H "User-Agent: $UA" "https://api.weather.gov/points/41.8458,-87.6272" | python3 -m json.
tool | head -40
{
    "@context": [
        "https://geojson.org/geojson-ld/geojson-context.jsonld",
        {
            "@version": "1.1",
            "wx": "https://api.weather.gov/ontology#",
            "s": "https://schema.org/",
            "geo": "http://www.opengis.net/ont/geosparql#",
            "unit": "http://codes.wmo.int/common/unit/",
            "@vocab": "https://api.weather.gov/ontology#",
            "geometry": {
                "@id": "s:GeoCoordinates",
                "@type": "geo:wktLiteral"
            },
            "city": "s:addressLocality",
            "state": "s:addressRegion",
            "distance": {
                "@id": "s:Distance",
                "@type": "s:QuantitativeValue"
            },
            "bearing": {
                "@type": "s:QuantitativeValue"
            },
```

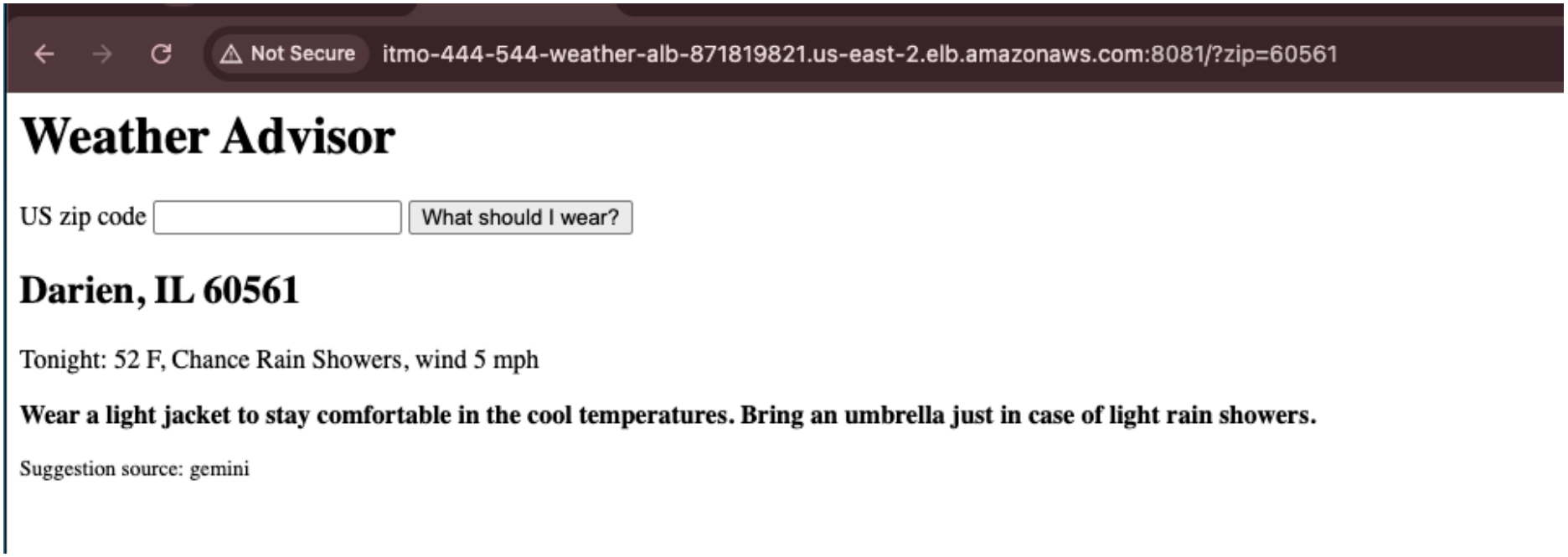
```
            "@type": "@id"
        },
        "forecastOffice": {
            "@type": "@id"
        },
        "forecastGridData": {
            "@type": "@id"
        },
        "publicZone": {
            "@type": "@id"
        },
        "county": {
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# FORECAST_URL=$(curl -s -H "User-Agent: $UA" "https://api.weather.gov/points/41.8458,-87.6272" \ |
 python3 -c "import sys,json; print(json.load(sys.stdin)['properties']['forecast'])") curl -s -H "User-Agent: $UA" "$FORECAST_URL" | python3 -m jso
n.tool | head -40
Expecting value: line 1 column 1 (char 0)
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# FORECAST_URL=$(curl -fsS -H "User-Agent: $UA" "https://api.weather.gov/points/41.8458,-87.6272" |
 python3 -c "import sys,json; print(json.load(sys.stdin)['properties']['forecast'])")
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -fsS -H "User-Agent: $UA" "$FORECAST_URL" | python3 -m json.tool | head -40
{
    "@context": [
        "https://geojson.org/geojson-ld/geojson-context.jsonld",
        {
            "@version": "1.1",
            "wx": "https://api.weather.gov/ontology#",
            "geo": "http://www.opengis.net/ont/geosparql#",
            "unit": "http://codes.wmo.int/common/unit/",
            "@vocab": "https://api.weather.gov/ontology#"
        }
    ],
    "type": "Feature",
    "geometry": {
        "type": "Polygon",
        "coordinates": [
            [
                [
                    -87.6047,
                    41.8435
                ],
                [
                    -87.6031,
                    41.8653
                ],
                [
                    -87.6324,
```

The first NWS API call does not return forecast data directly; it translates the latitude/longitude into the correct forecast endpoint and returns a URL for that forecast, so a second API call is required to retrieve the actual weather forecast.

Output of create_database.sh, and the Step 4 output showing retention 0, not public, single-AZ, and empty snapshot and automated backup lists.

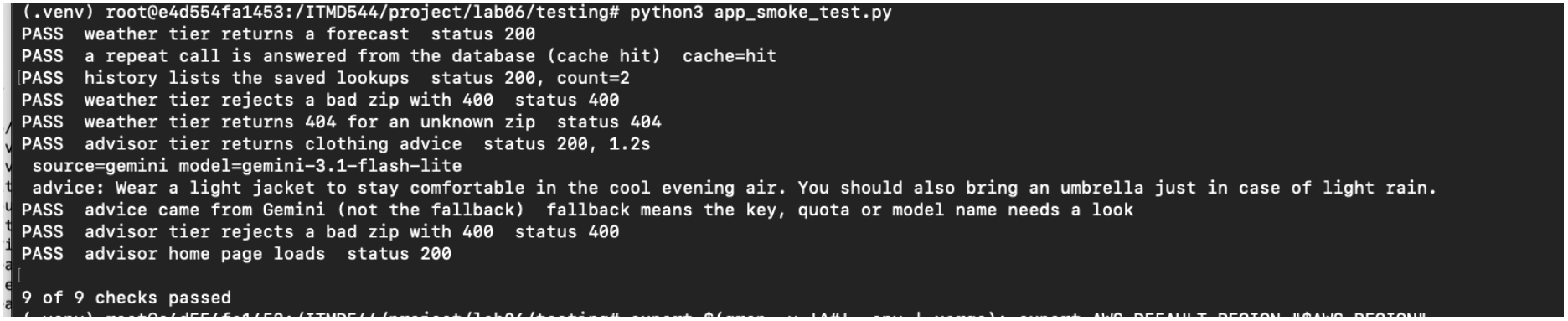


Screenshot of the advisor page showing a suggestion for a zip code, and the JSON from /advice?zip=... for the same zip.

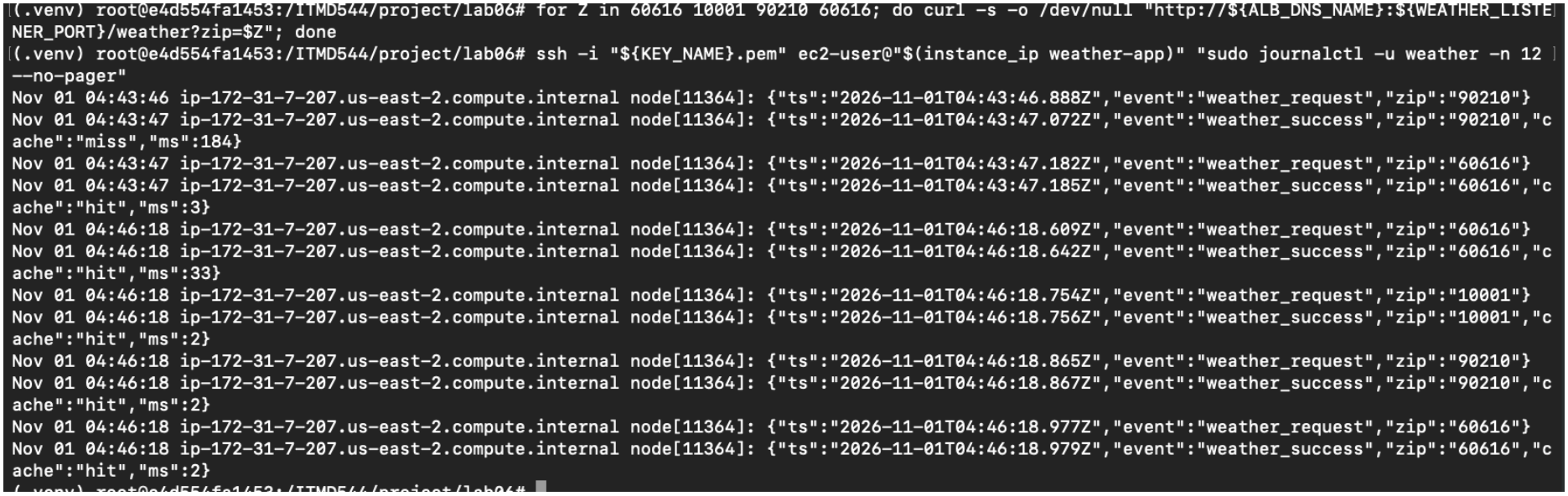

Not Secure
itmo-444-544-weather-alb-871819821.us-east-2.elb.amazonaws.com:8081/?zip=60561
Weather Advisor
US zip code
What should I wear?
Darien, IL 60561
Tonight: 52 F, Chance Rain Showers, wind 5 mph
Wear a light jacket to stay comfortable in the cool temperatures. Bring an umbrella just in case of light rain showers.
Suggestion source: gemini

```
curl -sS "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" | python3 -m json.tool
{
    "zip": "60616",
    "city": "Chicago",
    "state": "IL",
    "latitude": "41.8426",
    "longitude": "-87.6306",
    "forecast": {
        "name": "Tonight",
        "temperature": 54,
        "temperatureUnit": "F",
        "shortForecast": "Chance Rain",
        "windSpeed": "5 to 10 mph",
        "precipitationChance": 29,
        "isDaytime": false
    },
    "cache": "miss"
}
{
    "zip": "60616",
    "city": "Chicago",
    "state": "IL",
    "forecast": {
        "name": "Tonight",
        "isDaytime": false,
        "windSpeed": "5 to 10 mph",
        "temperature": 54,
        "shortForecast": "Chance Rain",
        "temperatureUnit": "F",
        "precipitationChance": 29
    },
    "latitude": "41.8426",
    "longitude": "-87.6306",
    "cache": "hit"
}
{
    "zip": "60616",
    "city": "Chicago",
    "state": "IL",
    "forecast": {
        "name": "Tonight",
        "isDaytime": false,
        "windSpeed": "5 to 10 mph",
        "temperature": 54,
```

Output of app_smoke_test.py with all nine checks passing.



Step 1: a few lines of the weather journal, with a sentence explaining one weather_success line.



Step 2: the /history output and the psql query output.

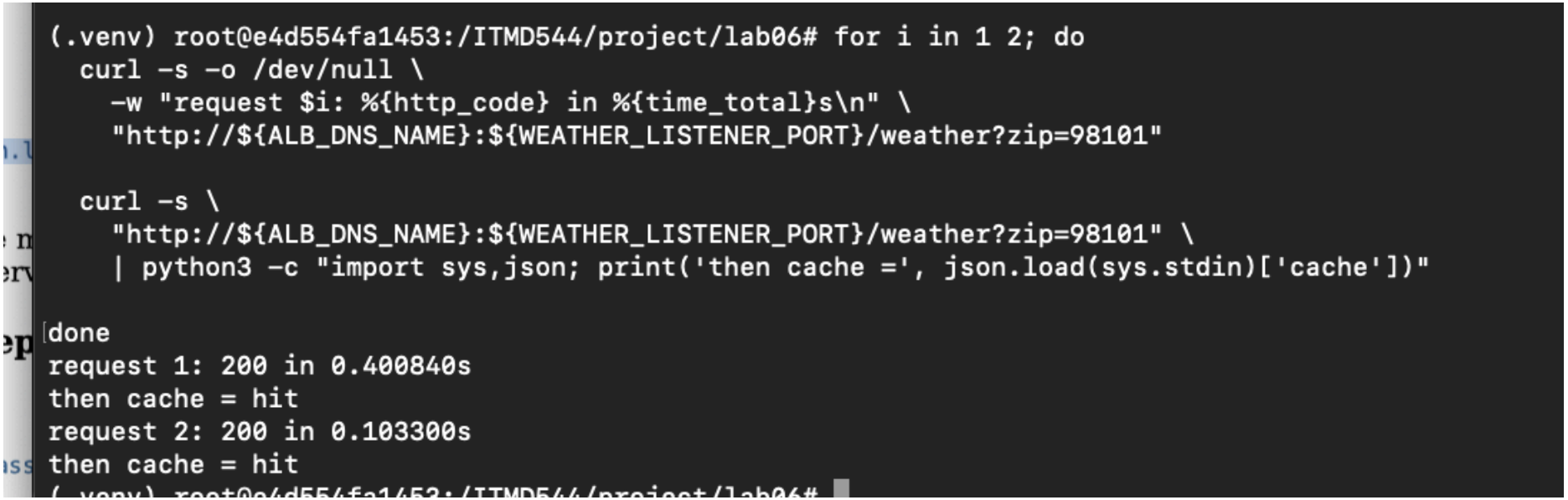
```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/history" | python3 -m json.tool
{
    "count": 3,
    "lookups": [
        {
            "id": "3",
            "zip": "90210",
            "city": "Beverly Hills",
            "state": "CA",
            "short_forecast": "Rain",
            "temperature": "59",
            "created_at": "2026-11-01T04:43:47.070Z"
        },
        {
            "id": "2",
            "zip": "10001",
            "city": "New York City",
            "state": "NY",
            "short_forecast": "Partly Cloudy",
            "temperature": "44",
            "created_at": "2026-11-01T04:43:46.700Z"
        },
        {
            "id": "1",
            "zip": "60616",
            "city": "Chicago",
            "state": "IL",
            "short_forecast": "Chance Rain",
            "temperature": "54",
            "created_at": "2026-11-01T04:43:46.211Z"
        }
    ]
}
```

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip weather-app)" \
  "PGPASSWORD=\$(aws ssm get-parameter \
    --name $DB_PASSWORD_PARAM \
    --with-decryption \
    --query Parameter.Value \
    --output text \
    --region $AWS_REGION) \
  psql 'host=$DB_HOST \
        dbname=$DB_NAME \
        user=$DB_USER \
        sslmode=verify-full \
        sslrootcert=/opt/app/rds-ca.pem' \
    -c 'SELECT id, zip, created_at FROM lookups ORDER BY id DESC LIMIT 5' \
    -c 'SELECT count(*) AS saved_lookups FROM lookups'"
 id |  zip  |          created_at
----+-------+-------------------------------
  3 | 90210 | 2026-11-01 04:43:47.070112+00
  2 | 10001 | 2026-11-01 04:43:46.700991+00
  1 | 60616 | 2026-11-01 04:43:46.211966+00
(3 rows)

 saved_lookups
---------------
             3
(1 row)

(.venv) root@e4d554fa1453:/ITMD544/project/lab06#
```

Step 3: the miss and hit timings, and your two sentences.



A short cache reduces calls to a rate-limited upstream weather API, helping prevent rate-limit errors and conserve API quota.

CACHE_SECONDS should not be too long because weather conditions change frequently, and users need reasonably fresh, up-to-date forecasts.

Step 5: the IAM simulator results for both roles and both secrets, and your sentence about the advisor tier.

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# GEMINI_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${GEMINI_PARAM_NAME}"
DB_ARN="arn:aws:ssm:${AWS_REGION}:${ACCOUNT_ID}:parameter${DB_PASSWORD_PARAM}"

for ROLE in "$WEATHER_ROLE_NAME" "$ADVISOR_ROLE_NAME"; do

  ROLE_ARN=$(aws iam get-role \
    --role-name "$ROLE" \
    --query Role.Arn \
    --output text)

  for P in "$DB_ARN" "$GEMINI_ARN"; do

    printf "%-40s %-50s " "$ROLE" "${P##*parameter}"

    aws iam simulate-principal-policy \
      --policy-source-arn "$ROLE_ARN" \
      --action-names ssm:GetParameter \
      --resource-arns "$P" \
      --query "EvaluationResults[0].EvalDecision" \
      --output text

  done
done
allowed4-544-weather-role               /itmo-444-544/db-password
implicitDeny-weather-role               /itmo-444-544/gemini-api-key
implicitDeny-advisor-role               /itmo-444-544/db-password
allowed4-544-advisor-role               /itmo-444-544/gemini-api-key
(.venv) root@e4d554fa1453:/ITMD544/project/lab06#
```

If an attacker compromises the advisor tier, they can access the Gemini API key because that role is allowed to read that SSM parameter, but they cannot access the database password because the IAM policy returns implicitDeny for the database-credential parameter, limiting the attacker's access to AI-service credentials rather than the application database.

Step 6: the direct and through-the-ALB results, and your sentence about security group sources.



A security group rule that uses another security group as its source allows access only from resources that are members of that specific security group, whereas a CIDR range allows access from any IP address within the specified network range and cannot enforce access based on workload identity or AWS-managed group membership.

Step 7: the output while the database was cut off, the db_error log line, and your sentence about the 30 second pause.

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ec2 revoke-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
> --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID"
{
    "Return": true,
    "RevokedSecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-003d2652900f6d506",
            "GroupId": "sg-026354abfce16578e",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 5432,
            "ToPort": 5432,
            "ReferencedGroupId": "sg-0874a6c67b792ea83"
        }
    ]
}
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip weather-app)" "sudo systemctl restart weather"
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# sleep 5
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,
json; d=json.load(sys.stdin); print(d['city'], 'cache =', d['cache'])"
Chicago cache = unavailable
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s -o /dev/null -w "history while the database is cut off: %{http_code}\n" "http://${ALB_DNS_NAM
E}:${WEATHER_LISTENER_PORT}/history"
history while the database is cut off: 503
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ec2 authorize-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
> --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID"
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-0f65b65a35488e6db",
            "GroupId": "sg-026354abfce16578e",
            "GroupOwnerId": "786589010080",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 5432,
            "ToPort": 5432,
            "ReferencedGroupInfo": {
                "GroupId": "sg-0874a6c67b792ea83",
                "UserId": "786589010080"
            },
            "SecurityGroupRuleArn": "arn:aws:ec2:us-east-2:786589010080:security-group-rule/sgr-0f65b65a35488e6db"
        }
```

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ec2 authorize-security-group-ingress --group-id "$DB_SECURITY_GROUP_ID" \
> --protocol tcp --port 5432 --source-group "$APP_SECURITY_GROUP_ID"
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-0f65b65a35488e6db",
            "GroupId": "sg-026354abfce16578e",
            "GroupOwnerId": "786589010080",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 5432,
            "ToPort": 5432,
            "ReferencedGroupInfo": {
                "GroupId": "sg-0874a6c67b792ea83",
                "UserId": "786589010080"
            },
            "SecurityGroupRuleArn": "arn:aws:ec2:us-east-2:786589010080:security-group-rule/sgr-0f65b65a35488e6db"
        }
    ]
}
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# sleep 35

(.venv) root@e4d554fa1453:/ITMD544/project/lab06#
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,
json; print('cache =', json.load(sys.stdin) ['cache'])"
cache = miss
(.venv) root@e4d554fa1453:/ITMD544/project/lab06#
```

the 30-second pause exists to allow the cache entry to expire so the application is forced to test its behavior when the database is unavailable; without that pause, every request would continue to be served from the cached result and would not exercise the database failure path, masking the outage and preventing verification that the application's fallback/error-handling logic works correctly.

Step 8: the fallback answers, the gemini_error journal lines, and your sentence about the weakness of silent fallback.

```
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# curl -s "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" | python3 -c "import sys,
json; print('cache =', json.load(sys.stdin) ['cache'])"
cache = miss
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ssm put-parameter \
  --name "$GEMINI_PARAM_NAME" \
  --type SecureString \
  --value "not-a-real-key" \
  --overwrite

sleep 70  # wait for the 60-second key cache to expire

for i in 1 2 3; do
  curl -s \
    "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=60616" \
  | python3 -c "
import sys, json
d = json.load(sys.stdin)
print(d['source'], '-', d['advice'])
"
done
{
    "Version": 2,
    "Tier": "Standard"
}
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
fallback - Wear a light jacket or a sweater over long sleeves. Bring an umbrella or a waterproof jacket.
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# ssh -i "${KEY_NAME}.pem" ec2-user@"$(instance_ip advisor-app)" \
> "sudo journalctl -u advisor -n 20 --no-pager | grep gemini_error"
The authenticity of host '3.12.155.19 (3.12.155.19)' can't be established.
ED25519 key fingerprint is SHA256:7VSkQwcZLrLDaYjbmpJTey4sONwoWIjPR+3YiWjvV3k.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.12.155.19' (ED25519) to the list of known hosts.
Nov 01 05:05:39 ip-172-31-1-37.us-east-2.compute.internal node[10608]: {"ts":"2026-11-01T05:05:39.340Z","event":"gemini_error","zip":"60616","status":4
00,"reason":"gemini_http_error"}
Nov 01 05:05:39 ip-172-31-1-37.us-east-2.compute.internal node[10608]: {"ts":"2026-11-01T05:05:39.555Z","event":"gemini_error","zip":"60616","status":4
00,"reason":"gemini_http_error"}
Nov 01 05:05:39 ip-172-31-1-37.us-east-2.compute.internal node[10608]: {"ts":"2026-11-01T05:05:39.713Z","event":"gemini_error","zip":"60616","status":4
00,"reason":"gemini_http_error"}
(.venv) root@e4d554fa1453:/ITMD544/project/lab06#
```

A silent fallback improves availability for users, but it can hide production failures because requests continue to succeed even when the Gemini integration is broken, making monitoring and alerting essential to detect the underlying problem.

Step 9: the recovery time you measured, what the advisor returned during the outage, and your sentence about where state should live. One paragraph on why the secrets are in SSM and each is read by one role, instead of in .env or the user-data.

```
aws autoscaling terminate-instance-in-auto-scaling-group \
  --instance-id "$VICTIM" \
  --no-should-decrement-desired-capacity

START=$(date +%s)

# In a second terminal, ask the advisor while the weather tier is down:
# curl -s -w "\n%{http_code}\n" \
#   "http://${ALB_DNS_NAME}:${ADVISOR_LISTENER_PORT}/advice?zip=33101"

until [ "$(
  curl -s -m 5 -o /dev/null -w '%{http_code}' \
    "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616"
)" = "200" ]; do
  sleep 10
done

echo "Weather service recovered after $(( $(date +%s) - START )) seconds"

curl -s \
  "http://${ALB_DNS_NAME}:${WEATHER_LISTENER_PORT}/weather?zip=60616" \
  | python3 -c "
import sys, json
print('cache =', json.load(sys.stdin)['cache'])
"
{
    "Activity": {
        "ActivityId": "b6c68656-1a63-3925-ba2b-c26555293584",
        "AutoScalingGroupName": "itmo-444-544-weather-asg",
        "Description": "Terminating EC2 instance: i-0d41a717f6e362779",
        "Cause": "At 2026-11-01T05:13:08Z instance i-0d41a717f6e362779 was taken out of service in response to a user request.",
        "StartTime": "2026-11-01T05:13:08.558000+00:00",
        "StatusCode": "InProgress",
        "Progress": 0,
        "Details": "{\"Availability Zone ID\":\"use2-az1\",\"Subnet ID\":\"subnet-0c7e269d21fe070c9\",\"Availability Zone\":\"us-east-2a\"}"
    }
}

Weather service recovered after 88 seconds
cache = hit
(.venv) root@e4d554fa1453:/IIMD544/project/lab06/testing#
```

Sentence about where state should live:

Application state should live in durable shared services such as databases, caches, or object storage rather than on individual EC2 instances, because instances can be terminated and replaced at any time by Auto Scaling without losing application data.

Paragraph on secrets in SSM:

Secrets are stored in AWS Systems Manager Parameter Store instead of `.env` files or EC2 user-data because Parameter Store provides centralized secret management, encryption, auditing, and fine-grained IAM access control. By giving the weather role permission to read only the database password and the advisor role permission to read only the Gemini API key, the system follows the principle of least privilege and limits the impact of a compromise. If secrets were embedded in `.env` files or user-data, they would be stored on the instance, be harder to rotate, and could expose unrelated credentials to an attacker who gains access to a server. Using separate SSM parameters and role-specific permissions ensures that each service receives only the secret it requires and nothing more.

One paragraph on turning backups off: what it saved, what it cost you, and when you would never do it.

Turning automated backups off saved cost by eliminating backup storage and retention charges, which can be useful in a short-lived lab environment where the database and its data can be recreated if lost. The tradeoff was a complete loss of point-in-time recovery and the ability to restore the database to an earlier state after accidental deletion, corruption, failed deployments, or other operational mistakes. In a production system that stores important business, customer, financial, or compliance-related data, I would never disable backups because the money saved is insignificant compared to the potentially catastrophic impact of permanent data loss and extended downtime.

One paragraph on what data you sent to Gemini and why the free tier makes that matter.

The application sent Gemini only the information needed to generate weather advice, such as the ZIP code and the weather data returned by the weather service, rather than database credentials, internal infrastructure details, or other sensitive application data. This matters especially on the free tier because requests may be subject to stricter quotas, rate limits, and provider data-handling policies, so minimizing the amount of information sent reduces privacy risk, lowers unnecessary token consumption, and limits the impact if the external AI service is unavailable or misconfigured.

Answers to the Section 15.1 review questions.

5.1 Review Questions

1. Secret storage - Where else could each leak from (shell history, process lists, logs, screenshots, Git)?
Even when secrets are stored in SSM SecureString, they can still leak through command-line history, environment variables visible to processes, application logs, debugging output, screenshots of terminals or AWS consoles, copied configuration files, CI/CD logs, and accidentally committed files in Git repositories. Secure storage reduces risk, but operational practices must also prevent secrets from being exposed elsewhere.

2. Least privilege - What would change if both roles had ssm:* on *?
If both roles had unrestricted access to all SSM parameters, compromising either service would give an attacker access to every stored secret, including credentials unrelated to that service. This would eliminate the security isolation provided by least privilege and significantly increase the impact of a breach.

3. Network exposure - Why is the ALB open to the internet but the instances and database are not?
The Application Load Balancer must be publicly reachable because it serves user requests, while the EC2 instances and database should remain private so they can only be accessed through controlled tiers. This reduces the attack surface and prevents direct access to internal application components.

4. Shared security group - What can an advisor instance do that it should not be able to do? How would you fix it?
Because both application tiers share the same security group and the database allows that security group, an advisor instance can connect directly to the database even if it does not need database access. The fix is to use separate security groups for the weather and advisor services and allow database access only from the security group used by the weather service.

5. Database login - What would you create instead, and which privileges would it need?
Instead of logging in as the database master user, I would create a dedicated application user with only the permissions required by the application. For this lab, the account would need permissions such as SELECT and INSERT on the lookups table but not administrative privileges like creating users, altering schemas, or dropping tables.

6. Encryption - What does the certificate check protect against that TLS without it does not?
TLS certificate validation confirms that the client is communicating with the legitimate database server rather than an impostor. Without certificate verification, an attacker could perform a man-in-the-middle attack by presenting a fraudulent certificate and intercepting encrypted traffic.

7. SQL injection - Show one line of the app and say what would go wrong if it built SQL by joining strings.
Using placeholders such as:

Python
```
cursor.execute(
"INSERT INTO lookups (zip) VALUES (%s)",
(zip_code,)
)
```

prevents user input from becoming executable SQL. If the query were built by string concatenation instead, a malicious user could inject SQL commands that read, modify, or delete database data.

8. Backups - What would you lose if this database held real data, and what would you configure instead?
With backups disabled, a database failure, accidental deletion, or corruption would permanently destroy all stored data. In production, I would enable automated backups, retention policies, and point-in-time recovery so the database could be restored after an incident.

9. Data lifecycle - Who decides when stored data may be destroyed in a real organization?
Data retention and deletion policies are normally determined by business owners, legal and compliance teams, security staff, and organizational governance requirements. Data should only be destroyed according to approved policies and regulatory obligations.

10. Untrusted output - What could an attacker gain if the page printed raw model output?
If AI-generated text were rendered without escaping, an attacker could potentially inject malicious HTML or JavaScript into the page, leading to cross-site scripting (XSS), session theft, credential theft, or unauthorized actions performed in a user's browser.

11. Data sent to a third party - What would you have to change before using a paid or school-owned Gemini account for real users?
Before using a production Gemini account, I would review privacy requirements, obtain any necessary user consent, establish a data-processing agreement, minimize transmitted data, implement monitoring and logging controls, and ensure compliance with organizational and regulatory policies.

12. Abuse and cost - How could someone use up your quota, and what could you add?
Because the advisor endpoint is publicly accessible, an attacker could generate large numbers of requests and exhaust the Gemini quota or incur unnecessary costs. Mitigations include rate limiting, AWS WAF rules, authentication, quotas per user, request throttling, and monitoring with alerts.

13. Availability - Why is it important that the response says source: fallback or cache: unavailable?
The response should clearly indicate whether data came from a cache or fallback mechanism so users and operators know they are not seeing normal live results. Transparency improves troubleshooting, monitoring, and operational awareness and helps prevent silent failures from going unnoticed.

Output of the Step 10 verification commands showing no database instance, snapshot, automated backup, load balancer or lab role remains.

```
based policy allows the ssm:DeleteParameter action
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws rds describe-db-instances --query "DBInstances[].DBInstanceIdentifier"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws rds describe-db-snapshots --query "DBSnapshots[].DBSnapshotIdentifier"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws rds describe-db-instance-automated-backups --query "DBInstanceAutomatedBackups[].DBInstanceIdenti
fier"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws rds describe-db-subnet-groups --query "DBSubnetGroups[].DBSubnetGroupName"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws elbv2 describe-load-balancers --query "LoadBalancers[].LoadBalancerName"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws autoscaling describe-auto-scaling-groups --query "AutoScalingGroups[].AutoScalingGroupName"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ec2 describe-launch-templates --query "LaunchTemplates[].LaunchTemplateName"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws iam list-roles --query "Roles[?starts_with(RoleName, 'itmo-444-544')].RoleName"
[]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ssm describe-parameters --query "Parameters[].Name"
[
    "/itmo-444-544/gemini-api-key"
]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ssm describe-parameters --query "Parameters[].Name"
[
    "/itmo-444-544/gemini-api-key"
]
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ssm delete-parameter --name "$GEMINI_PARAM_NAME"
(.venv) root@e4d554fa1453:/ITMD544/project/lab06# aws ssm describe-parameters --query "Parameters[].Name"
[]
```